

第12週: 状態を管理する (React State & Events)

～ 「静的」から「インタラクティブ」へ～

1. 状態 (State) とは？

アプリが覚えている「今の値」のこと。

- カウンターの数字。
- 入力欄に入力中の文字。
- ダークモードかどうか。

Vanilla JS: DOM を直接書き換えていた。

React: State を更新すると、React が自動で見た目を書き換えてくれる。

2. **useState** フック

値を保存し、それを更新するための「関数」を提供してくれます。

```
'use client'; // インタラクティブな機能には必須！
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <button onClick={() => setCount(count + 1)}>
      Like: {count}
    </button>
  );
}
```

3. 'use client' (Client Components)

Next.js (App Router) では、デフォルトでは「サーバー側」で処理されます。

- クリックイベントや State を使うには、ファイルの先頭に `'use client';` と書く必要があります。
- これにより、ブラウザ側で動くコードであることを宣言します。

4. 本日の目標: ロジックの復元

Phase 2 で作った機能を React の State で書き直します。

1. **Like Counter:** ボタンを押すと数字が増える。
2. **Status Update:** 入力欄の文字がプロフィール文に反映される。
3. **Theme Toggle (Bonus):** ダークモードの切り替え。

5. ステート・リフティング (State Lifting)

「入力欄 (Component A)」の値を「プロフィール (Component B)」に伝えたい場合。

- **共通の親要素** で State を持ち、それを Props として配るのが React の基本パターンです。

宿題 (Homework)

1. `useState` を使って、いいねボタンのカウンターを実装する。
2. ステータス更新フォームを作成し、入力内容がページ内に即座に反映されるようにする。
3. [発展] 背景色を切り替えるトグルボタンを実装してみる。